Table of contents

ABSTRACT

The Employee Management System (EMS) is a desktop-based application designed to efficiently manage employee records within an organization by replacing traditional manual methods with a digital solution. The system enables users to perform operations such as adding, updating, deleting, and searching employee information in a structured and secure manner using a relational database. Developed using Core Java, Swing for the graphical user interface, JDBC for database connectivity, and Oracle Database for backend storage, the application ensures data accuracy, reduces redundancy, and improves accessibility. With its user-friendly interface and efficient data handling capabilities, the system enhances organizational productivity and provides a reliable solution for managing employee information.

LIST OF FIGURES

LIST OF TABLES

ACRONYMS

- **EMS** : Employee Management System
- **JDBC** : Java Database Connectivity
- **GUI** : Graphical User Interface
- **SQL** : Structured Query Language

1. INTRODUCTION

1.1 Brief Information about the Project

The Employee Management System (EMS) is a desktop-based application developed to manage employee information efficiently within an organization. It provides a centralized platform to store, retrieve, update, and delete employee records in a structured manner. The system replaces traditional manual record-keeping methods with a computerized solution, thereby improving data accuracy, reducing redundancy, and enhancing accessibility. The application is developed using Core Java for programming, Swing for designing the graphical user interface, JDBC for database connectivity, and Oracle Database for storing employee data. It is designed to be user-friendly and suitable for use in organizations such as colleges, offices, and companies.

1.2 Motivation and Contribution of Project

In many organizations, employee data is still maintained using manual registers or basic spreadsheet tools, which leads to inefficiencies such as data duplication, difficulty in searching records, and increased chances of errors or data loss. These limitations motivated the development of the Employee Management System.

The key contributions of this project include:

- Providing a digital solution for managing employee records
- Reducing manual effort and paperwork
- Improving accuracy and consistency of data
- Enabling quick search and retrieval of employee information
- Enhancing data security through controlled access

This system simplifies employee data management and improves overall organizational efficiency.

1.3 Objectives of the Project

The main objectives of the Employee Management System are:

- To develop a user-friendly application for managing employee data
- To store employee information in a structured database
- To provide functionalities for adding, updating, and deleting employee records
- To enable quick searching of employee details using ID or name
- To display employee data in an organized format

1.4 Scope of the Project

The Employee Management System can be used in small and medium-sized organizations such as offices, colleges, and institutions to manage employee records efficiently. It supports operations such as adding new employee details, updating existing records, deleting employee information, searching for employees using ID or name, and displaying all records in an organized manner. The system provides a simple and user-friendly interface for maintaining employee data in a centralized database.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional requirements define the core operations and functionalities that the Employee Management System (EMS) must perform to effectively manage employee-related information. The Functional requirements of Employee Management System (EMS)

- **Add Employee Details**

  The system should allow the administrator to add new employee records, including details such as employee ID, name, designation, department, contact information, and salary details. Proper validation should be applied to ensure data accuracy.

- **Update Employee Records**

  The system should provide functionality to modify or update existing employee information whenever required. This includes editing personal details, job roles, department changes, or salary updates.

- **Delete Employee Information**

  The system should allow authorized users to delete employee records from the database. A confirmation mechanism should be included to prevent accidental deletion of important data.

- **Search Employee**

  The system should support searching for employees using specific criteria such as employee ID or employee name. The search functionality should return accurate and relevant results quickly.

- **Display Employee Records**

  The system should display all employee records in a structured format, enabling users to view complete details of employees. Sorting and filtering options can be included for better usability.

- **Manage Department Details**

  The system should allow management of department-related information, including adding, updating, and deleting department records. Employees should be associated with their respective departments.

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards that the **Employee Management System** must satisfy.

- **Performance**

  The system should respond quickly to operations like search, add, and update without delays.

- **Security**

  Only authorized users should access and modify data using proper authentication and access control.

- **Usability**

  The system should be simple and easy to use, allowing users to perform tasks without difficulty.

- **Reliability**

  The system should operate consistently and handle errors without data loss.

- **Scalability**

  The system should handle increasing data and users without affecting performance.

2.3 Requirements Specification

This section specifies the hardware and software needed to run the system.

2.3.1 Minimum Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB (minimum)
- Hard Disk: 20 GB free space
- Monitor: Standard display
- Keyboard and Mouse

2.3.2 Software Requirements

- **Operating System:** Windows 7/10/11
- **Programming Language:** Java
- **IDE:** VS Code
- **Database:** Oracle Database
- **Connectivity:** JDBC

3. TECHNOLOGY DESCRIPTION

3.1 Programming Language

3.1.1 Introduction to Java

Java is a high-level, object-oriented programming language developed by Sun Microsystems. It is widely used for developing desktop, web, and enterprise applications due to its platform independence and robustness.

In the Employee Management System, Java is used to implement the core functionality of the application. It handles operations such as adding, updating, deleting, and searching employee records. Java provides features like object-oriented programming, exception handling, and strong memory management, which help in building reliable and efficient applications.

One of the main advantages of Java is that it is platform-independent, meaning the program can run on any system that has a Java Virtual Machine (JVM). This makes it suitable for developing portable applications like the Employee Management System.

3.2 Framework / Libraries

3.2.1 Introduction to Swing

Swing is a part of Java that is used to create graphical user interfaces (GUI). It provides a set of components such as buttons, labels, text fields, tables, and frames to design user-friendly applications.

In this project, Swing is used to design the user interface of the Employee Management System. It allows users to interact with the system easily through windows such as:

- Login Window
- Add Employee Form
- Update Employee Form
- Search Employee Window
- Display Employee Records

Swing provides flexibility in designing layouts and improves the overall user experience. It makes the application visually interactive and easy to use.

3.3 Database Technology

The Employee Management System uses a relational database for storing and managing employee information in a structured manner. The database ensures efficient data storage, retrieval, and manipulation, which is essential for maintaining accuracy and

consistency in the system. In this project, Oracle Database is used as the backend, and JDBC is used for connecting the Java application to the database.

3.3.1 JDBC (Java Database Connectivity)

JDBC is an API (Application Programming Interface) used in Java to connect and interact with databases. It allows Java applications to execute SQL queries and retrieve or update data in the database.

In the Employee Management System, JDBC is used to connect the Java application with the Oracle database. It performs operations such as:

- Establishing a connection to the database
- Executing SQL queries (INSERT, UPDATE, DELETE, SELECT)
- Retrieving data from the database
- Closing the database connection

JDBC acts as a bridge between the application and the database, ensuring smooth communication and data transfer. It helps in maintaining data consistency and enables efficient data handling.

3.3.2 Oracle Database

Oracle Database is a powerful relational database management system used for storing and managing structured data. It provides high performance, reliability, and security for handling large volumes of data.

In the Employee Management System, Oracle Database is used to store employee details such as employee ID, name, department, designation, and contact information. It supports SQL for data manipulation and ensures data integrity through constraints like primary keys and data types.

3.4 Tools & IDEs

3.4.1 Visual Studio Code

Visual Studio Code (VS Code) is used as the development environment for the Employee Management System. It is a lightweight code editor that supports Java programming through extensions. VS Code provides features such as syntax highlighting, code completion, and debugging support, which help improve coding efficiency. It’s simple and user-friendly interface makes it suitable for developing and testing the application.

4. DATABASE DESIGN

4.1 Database Tables

The Employee Management System uses a relational database to store employee information in a structured manner. The database consists of two main tables: **Employee** and **Department**.

- The **Department** table stores details of different departments in the organization.
- The **Employee** table stores employee information and is linked to the Department table.

This separation of data helps in reducing redundancy and improves data organization.

4.2 Table Structure

4.2.1 Department Table

Field Name	Data Type	Description	Constraint
dept_id	NUMBER	Department ID	Primary Key
dept_name	VARCHAR2(50)	Department Name	Not Null

Table 1: Department Relational Database Table

4.2.2 Employee Table:

Field Name	Data Type	Description	Constraint
emp_id	NUMBER	Employee ID	Primary Key
emp_name	VARCHAR2(100)	Employee Name	Not Null
designation	VARCHAR2(50)	Job Role	-
salary	NUMBER	Salary	-
phone	VARCHAR2(15)	Contact Number	-
email	VARCHAR2(100)	Email Address	-
dept_id	NUMBER	Department ID	Foreign Key

Table 2: Employee Relational Database Table

4.3 Keys and Constraints

Keys and constraints are used to maintain data integrity and ensure accurate data storage.

- **Primary Keys:**
    - Emp_ID in Employee table

    - Dept_ID in Department table
- **Foreign Key:**
    - Dept_ID in the Employee table references Dept_ID in the Department table
- **NOT NULL Constraint:**
    - Important fields such as employee ID, name, and department ID cannot be left empty
- **Data Type Constraints:**
    - Each field accepts only valid data types to ensure correctness

These constraints help in maintaining consistency and avoiding duplicate or invalid data.

4.4 Table Relationships

The Employee and Department tables are related through the Dept_ID field.

- Each employee belongs to one department
- One department can have multiple employees

This represents a one-to-many relationship between Department and Employee tables.

5. SYSTEM DESIGN

5.1 System Architecture

The Employee Management System follows a **three-layer architecture**:

1. **Presentation Layer (User Interface)**
   - Developed using Java Swing
   - Allows users to interact with the system
2. **Application Layer (Business Logic)**
   - Processes user inputs
   - Performs operations like add, update, delete, search
3. **Data Layer (Database)**
   - Stores employee and department data
   - Managed using Oracle Database

This architecture ensures better organization, security, and maintainability.

5.2 Modules Description

The system is divided into the following modules:

1. Employee Management Module

- Adds new employee details
- Updates existing records
- Deletes employee data
- Shows employee data in table format

2. Department Module

- Stores department details
- Assigns employees to departments

3. Search Module

- Searches employee by ID or name

4. Database Connectivity Module

- Connects Java application with Oracle DB
- Executes SQL queries

5. User Interface Module

- Provides GUI using Swing
- Handles user interactions

6. SYSTEM IMPLEMENTATION

Login Screen:

Displays the login interface where the user enters username and password.

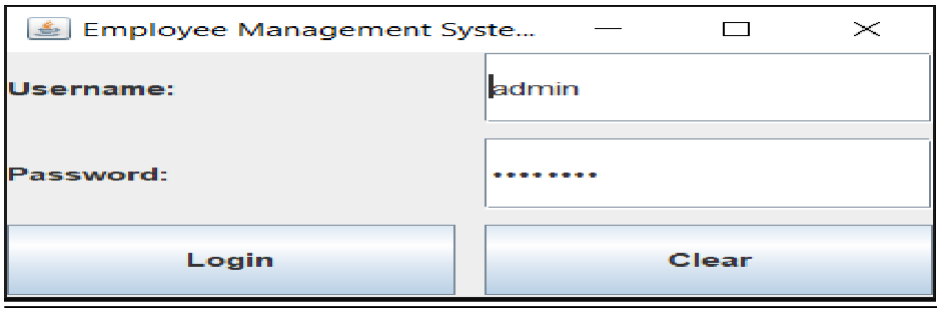

Figure 1: Login Screen of Employee Management System

Main Menu Screen:

Provides options such as Add, Update, Delete, Search, and View Employees.

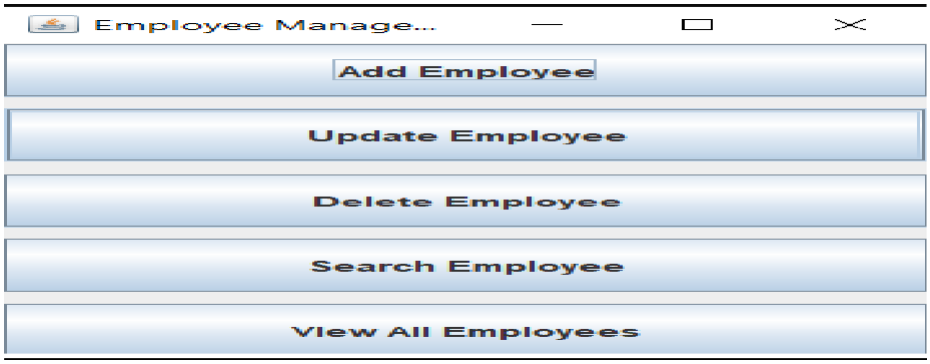

Figure 2: Main Menu Screen of Employee Management System

Add Employee Screen:

Shows the form used to enter new employee details.

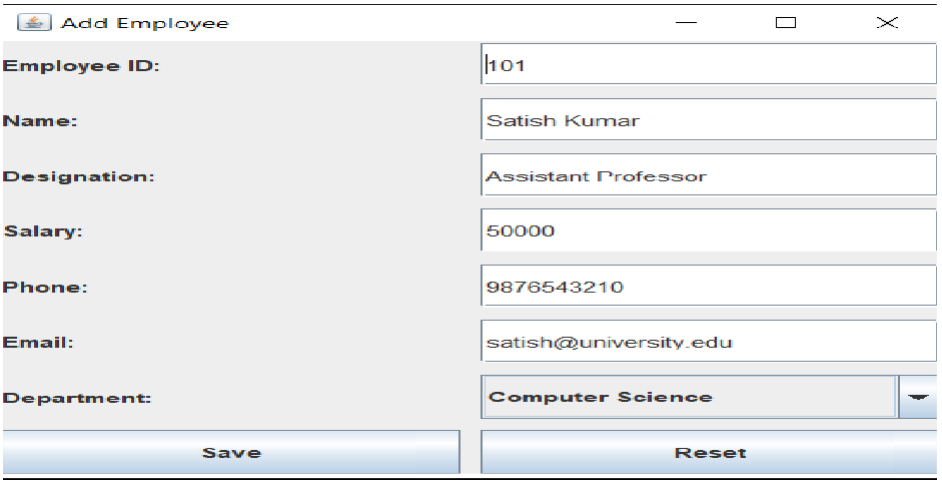

Figure 3: Employee Addition Screen of Employee Management System

Update Employee Screen:

Displays the interface for modifying existing employee details.

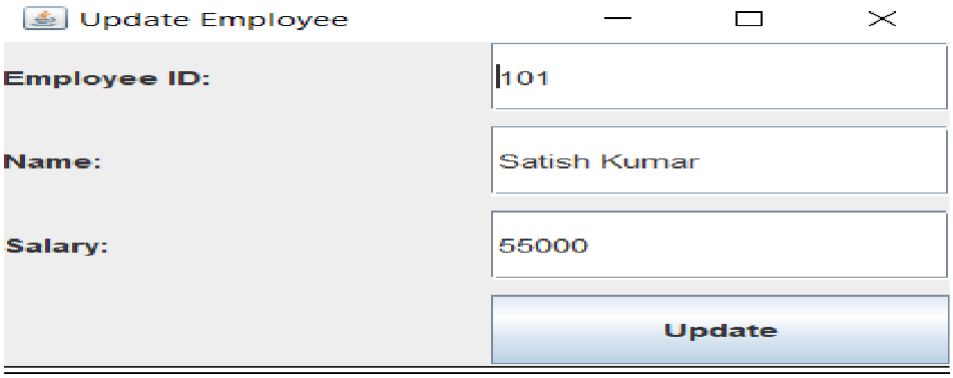

Figure 4: Employee Updation Screen of Employee Management System

View Employees Screen

Displays all employee records in a table format.

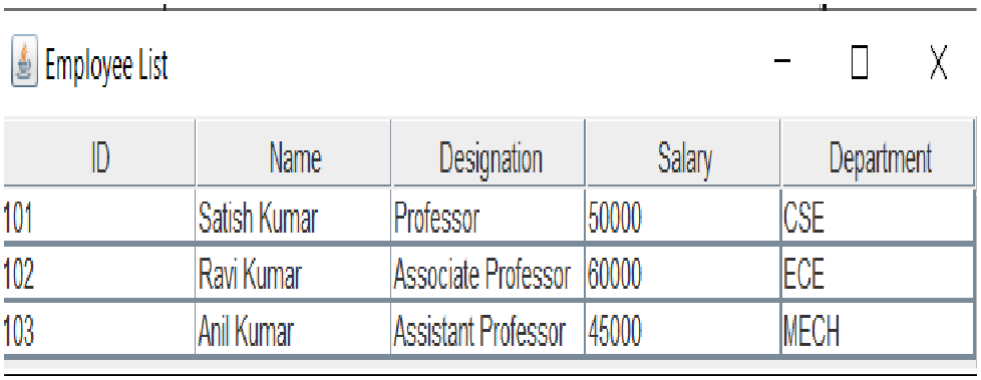

Figure 5: Employees Information Screen of Employee Management System

Delete Employee Screen

Shows the interface used to delete an employee record.

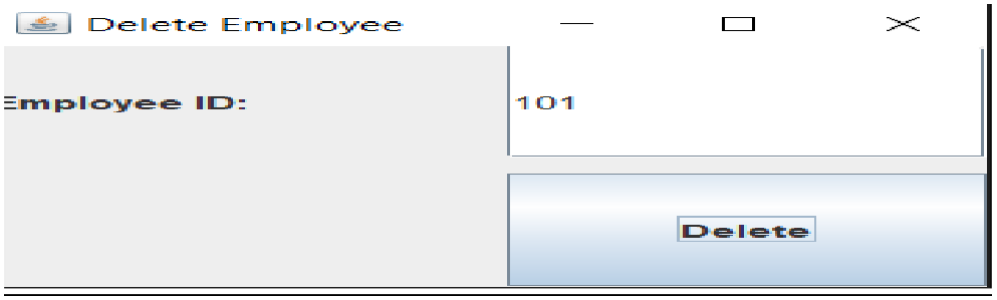

Figure 6: Employee Deletion Screen of Employee Management System

7. CONCLUSION

The Employee Management System has been successfully developed to manage employee information in an efficient and organized manner. The system replaces manual record-keeping with a computerized solution, thereby reducing errors, saving time, and improving data accuracy.

The application provides essential functionalities such as adding, updating, deleting, and searching employee records through a user-friendly interface. By using technologies like Java, Swing, JDBC, and Oracle Database, the system ensures reliable performance, secure data handling, and easy access to employee information.

Overall, the system improves the efficiency of managing employee data and reduces the workload of administrative staff. It also provides a scalable solution that can be further enhanced in the future by adding features such as web-based access, role-based authentication, and report generation.

In conclusion, the Employee Management System meets the project objectives and serves as an effective tool for managing employee records in organizations.

8. FUTURE ENHANCEMENTS

The Employee Management System developed in this project provides basic functionalities for managing employee records. However, the system can be further enhanced by adding advanced features to improve its functionality and usability.

Some possible future enhancements include:

- **Payroll Management:**
  Adding salary calculation, payslip generation, and tax management features
- **Attendance Management:**
  Integrating attendance tracking and leave management
- **Web-Based Application:**
  Converting the desktop application into a web-based system for remote access
- **Multi-User Support:**
  Allowing multiple users to access the system simultaneously
- **Improved User Interface:**
  Enhancing the design for better user experience

These enhancements can make the system more scalable, secure, and suitable for real-time organizational use.

9. REFERENCES

The following references were used during the development of the Employee Management System:

1. **Herbert Schildt**, *Java: The Complete Reference*, Oracle Press, 11th Edition.
2. **E. Balagurusamy**, *Programming with Java*, McGraw Hill Education.
3. **Korth, Silberschatz, Sudarshan**, *Database System Concepts*, McGraw Hill.
4. **Oracle Documentation**, https://docs.oracle.com
5. **Java Documentation (Oracle)**, https://docs.oracle.com/javase
6. **JDBC Tutorial – GeeksforGeeks**, https://www.geeksforgeeks.org/jdbc/
7. **Java Swing Tutorial – TutorialsPoint**, https://www.tutorialspoint.com/java_swing
8. **W3Schools Java Tutorial**, https://www.w3schools.com/java/

APPENDIX

A.1 Sample Code

Database Connection Code:

The following code establishes a connection between the Java application and the Oracle database using JDBC.

DBConnection.java:

```
import java.sql.*;
public class DBConnection {
   public static Connection getConnection() {
      Connection con = null;
      try {
         Class.forName("oracle.jdbc.driver.OracleDriver");
         con = DriverManager.getConnection(
            "jdbc:oracle:thin:@localhost:1521:xe","system","password");
      } catch (Exception e) {
         System.out.println(e);
      }
      return con;
   }
}
```

Insert Employee Record:

This code is used to add a new employee into the database.

InsertEmployee.java:

```
import java.sql.*;
public class InsertEmployee {
        public static void main(String[] args) {
         try {
        Connection con = DBConnection.getConnection();
        String query = "INSERT INTO employee
VALUES(?,?,?,?,?,?,?)";
        PreparedStatement pst = con.prepareStatement(query);
        pst.setInt(1, 101);
        pst.setString(2, "Ravi");
```

```
            pst.setString(3, "Manager");
            pst.setDouble(4, 50000);
            pst.setString(5, "9876543210");
            pst.setString(6, "ravi@gmail.com");
            pst.setInt(7, 1);
            int i = pst.executeUpdate();
            if (i > 0)
                System.out.println("Employee Added Successfully");
            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

A.2 SQL Queries

Create Department Table:

```
CREATE TABLE department (dept_id NUMBER PRIMARY KEY,
                         dept_name VARCHAR2(50) NOT NULL
                         );
```

Create Employee Table:

```
CREATE TABLE employee (emp_id NUMBER PRIMARY KEY,
                       emp_name VARCHAR2(100) NOT NULL,
                           designation VARCHAR2(50),
                           salary NUMBER,
                           phone VARCHAR2(15),
                           email VARCHAR2(100),
                           dept_id NUMBER,
                           FOREIGN KEY (dept_id)
                           REFERENCES department(dept_id)
                           );
```